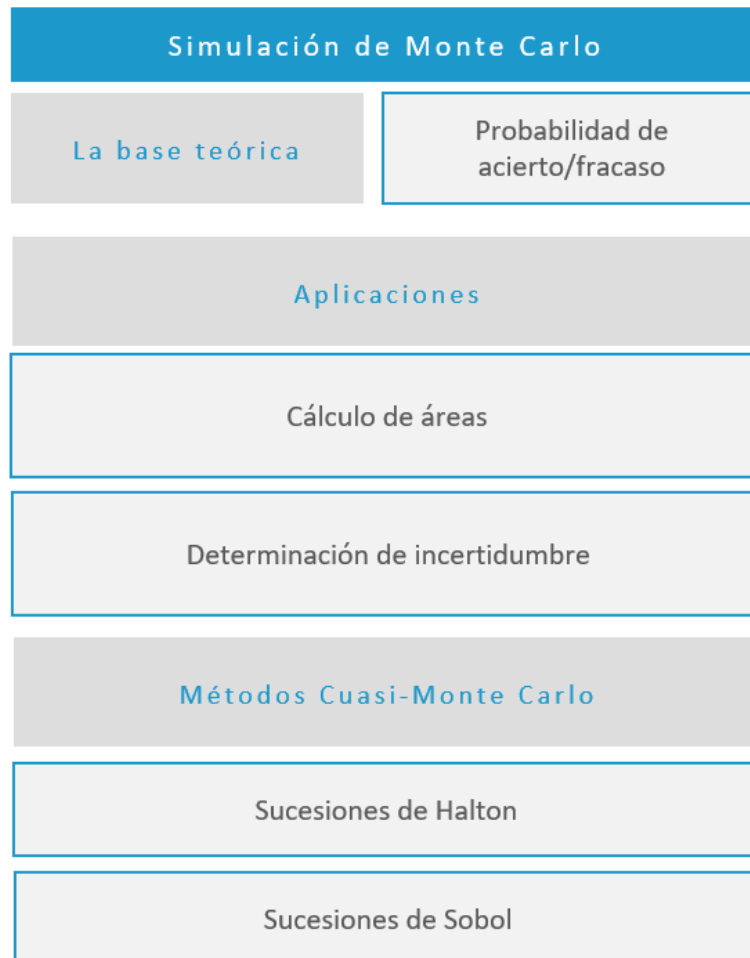


Modelado de Sistemas Dinámicos

Simulación de Monte Carlo

Índice

Esquema.	2
Ideas clave	3
8.1 Introducción y objetivos	3
8.2 La Simulación de Monte Carlo	6
8.3 Monte Carlo para el cálculo de áreas y volúmenes	9
8.4 Monte Carlo para el estudio de incertidumbre	16
8.5 Referencias bibliográficas	18



8.1 Introducción y objetivos

Hasta ahora hemos estudiado distintos sistemas y cómo modelarlos. También hemos realizado simulaciones de los sistemas para determinar la solución a problemas dados. Además, hemos estudiado como introducir valores aleatorios en el sistema. En este tema nos centraremos en un tipo concreto de simulación. La simulación de Monte Carlo.

Los métodos de Monte Carlo son métodos de simulación que se basan en el uso de valores aleatorios para estudiar un problema.

Definición 1

El método de Monte Carlo es una técnica de análisis cuantitativo que permite predecir errores y resolver problemas a partir de la obtención de valores aleatorios.

Ejemplo inicial

Queremos calcular el área de un círculo de radio 1 centrado en el origen. Para ello, vamos a realizar la siguiente estrategia. Vamos a generar un par de valores aleatorios, x_0 e y_0 uniformemente distribuidos en el intervalo $(-1, 1)$. A continuación miraremos si el punto (x_0, y_0) está dentro del círculo o no. Si está dentro del círculo añadiremos uno al contador de puntos en el círculo. Repetiremos este proceso n veces. La Figura 1 se muestra un ejemplo de $n = 100$ puntos.

Sabemos que el área del cuadrado es de $4u^2$, tendremos que la relación entre el área del círculo y el área del cuadrado será la misma que la relación entre el número de puntos que han caído dentro del círculo y el número n de puntos lanzados. Así pues, si A es el área del círculo y n_c es el número de puntos que han caído dentro del círculo tendremos que

$$\frac{A}{4} = \frac{n_c}{n}.$$

Por lo que $A = \frac{4n_c}{n}$.

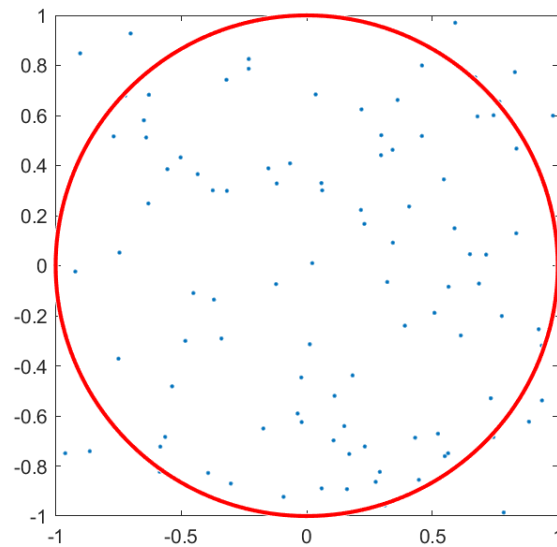


Figura 1: Conjunto de puntos aleatorios generados dentro y fuera del círculo.

El código siguiente genera una aproximación del área del círculo en Matlab.



T8eje0.m

```
n=100;
nc=0;

for i=1:n
    x=rand(2,1)*2-1;
    if(vecnorm(x)<1)
        nc=nc+1;
    end
end
A=4*nc/n
```

Sabemos que el área de un círculo de radio 1 es π . Así pues, podemos preguntarnos por el error que estamos cometiendo y cómo evoluciona con el número de iteraciones. La Figura 2 nos muestra el valor obtenido para el error con el número de iteraciones realizadas.

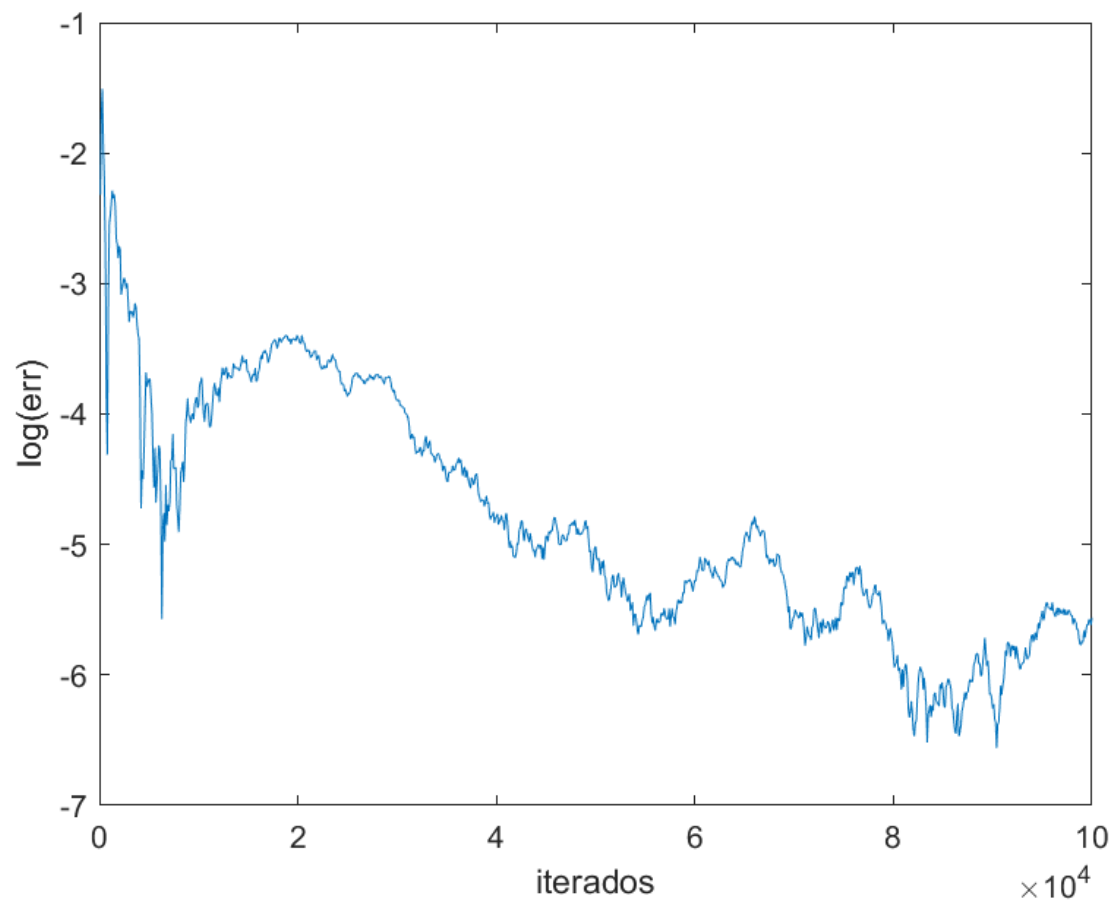


Figura 2: Relación de errores obtenidos en escala logarítmica en función del número de iterados

8.2 La Simulación de Monte Carlo

Los orígenes de la simulación están ligados al proyecto Monte Carlo desarrollado en el laboratorio de Los Álamos en el año 1940 por Stan Ulam y John Von Neumann (Metropolis and Ulam, 1949), en el que investigaban el movimiento aleatorio de los neutrones para la generación de la bomba atómica. La denominación de Monte Carlo proviene de la ciudad conocida por sus casinos al considerarse la ruleta uno de los primeros elementos físicos generador de números aleatorios.

Este método se ha usado y se usa en la actualidad para modelar una gran variedad de sistemas físicos y sistemas conceptuales. Proporciona un apoyo importante a la toma de decisiones con la aportación de los posibles resultados que se puede obtener cuando se lleva a cabo una acción y la probabilidad de que estos resultados se produzcan.

Además, permite la resolución de determinados problemas que no se podrían resolver por métodos exclusivamente numéricos o analíticos, al depender de elementos aleatorios, como por ejemplo el tiempo entre dos erupciones solares.

Estimación de las variables y del tamaño de la muestra

El método de Monte Carlo permite llevar a cabo un análisis de riesgos. Para ello, deberemos construir un modelo de simulación que ejecutaremos de forma iterativa. En cada iteración ejecutaremos un conjunto de valores aleatorios. La clave del método consiste en usar siempre la misma distribución inicial. Dependiendo de los datos y de que tenemos y de la variable aleatoria que los genera, la simulación requerirá de miles de nuevos datos para conseguir disminuir el grado de incertidumbre.

En cada repetición deberemos almacenar los valores obtenidos para poder concluir con fiabilidad estadística. Una vez tengamos los resultados de la simulación, tendremos una visión completa del sistema. Por lo tanto, el método no solo nos muestra el resultado o resultados que podemos esperar, sino también nos indica la probabilidad de que se den dichos resultados.

Así pues, en toda aplicación del método de Monte Carlo tendremos la intervención de números aleatorios. Así pues, necesitaremos emplear la generación de variables aleatorias que ya hemos visto en esta asignatura.

Para generar una buena resolución se requieren de un gran cantidad de datos. Así pues, el número de muestras necesarias para obtener el resultado final será muy importante. Esto ha hecho que no se emplearan de forma extensa hasta la explosión de los ordenadores. Gracias a esta explosión, y a la velocidad de cálculo, este tipo de métodos se han expandido mucho.

Así mismo, este gran número de iteraciones que son necesarias provoca que sea fundamental determinar el tamaño n de la muestra que debemos calcular para obtener una buena precisión en la solución del problema.

En las próximas líneas vamos a ver cómo aproxima el orden de mejora de los errores. Vamos a considerar el ejemplo inicial. Podemos realizar un cálculo parecido en otros casos.

Nuestro ejemplo inicial consiste en un generar un punto de forma aleatoria y determinar si está dentro de un círculo o no. Así pues, se trata de un experimento de tipo Bernoulli. Estos experimentos se caracterizan por tomar un valor de 1 con una probabilidad de éxito p y 0 con una probabilidad de $1 - p$. Tendremos 2 posibles valores, éxito y fracaso.

El recuento de los puntos que caen en el círculo, consiste en contar cuántos aciertos ha habido en n lanzamientos. Es decir, si X es una variable aleatoria de tipo Bernoulli, entonces $B = X_1 + \dots + X_n$, se trata de una variable aleatoria de tipo Binomial. Las distribución binomial calcula la probabilidad de obtener k aciertos en n lanzamientos. La función de probabilidad viene dada por:

$$P(X = k) = \binom{n}{k} p^k (1 - p)^{n-k}$$

A partir de la función de probabilidad podemos calcular la media y la varianza de la distribución binomial vienen dadas por:

$$E[X] = np,$$

$$\text{Var}[X] = np(1 - p).$$

Por lo que la desviación típica quedará $\sigma = \sqrt{np(1 - p)}$.

Sabemos que la probabilidad de acertar es $p = A/S$, con $S = 4$ el área del cuadrado.

Por otro lado, podemos aproximar $A = S \cdot \frac{n_c}{n}$. Con lo que nos quedará la desviación típica:

$$\sigma = \sqrt{n \cdot S \cdot \frac{n_c}{nS} \left(1 - \frac{n_c}{n}\right)} = \sqrt{n_c - \frac{n_c^2}{n}}$$

La desviación típica nos proporciona una aproximación del error del método. Si dividimos la desviación por el número de muestras obtenemos una aproximación del error relativo. Por lo que tendremos un error relativo:

$$e_r = \frac{\sigma}{n} = \frac{1}{n} \sqrt{n_c - \frac{n_c^2}{n}}$$

Ahora, supongamos que multiplicamos el número de lanzamientos por un factor m . Esto implica que tanto n como n_c se verán incrementados por m . Así pues, tendremos que el error de este segundo caso será:

$$e'_r = \frac{1}{nm} \sqrt{n_cm - \frac{n_c^2 m^2}{nm}} = \frac{1}{n} \sqrt{\frac{n_c}{m} - \frac{n_c^2}{nm}}$$

Ahora, para saber qué efecto ha tenido en el error dividimos ambos errores:

$$\frac{e_r}{e'_r} = \frac{\frac{1}{n} \sqrt{n_c - \frac{n_c^2}{n}}}{\frac{1}{nm} \sqrt{\frac{n_c}{m} - \frac{n_c^2}{nm}}} = \frac{1}{\sqrt{m}}.$$

Por lo tanto, multiplicar la precisión por m implica dividir el error por $\frac{1}{\sqrt{m}}$.

Podemos concluir que el error del método Monte Carlo es del orden de $\frac{1}{\sqrt{n}}$, lo que implica que por cada cifra decimal que se quiera ganar en la precisión, el tamaño de la muestra se debe de multiplicar por 100. Por tanto, cuanto mayor sea el tamaño de la muestra se obtendrán mejores resultados.

Aplicaciones del método

Desde sus inicios, la simulación de Monte Carlo a presentado multitud de aplicaciones. Entre las principales se encuentran:

- ▶ La representación gráfica de los datos y las probabilidades de que ocurran, lo que facilita la comunicación de los resultados.
- ▶ Realización de un análisis de sensibilidad determinando qué datos de entrada producen un mayor efecto en los resultados finales.
- ▶ Realización de un escenario combinando valores de distintas entradas para comprobar los efectos que producen.
- ▶ Establecer la correlación de entradas, es decir, las relaciones entre los datos de entrada.

De entre todas las posibles aplicaciones, vamos a estudiar dos de las más elementales. La primera de ellas ya la hemos apuntado en el ejemplo inicial, **el cálculo de integrales**. En especial vamos a calcular áreas y volúmenes mediante el método de Monte Carlo. La segunda de las aplicaciones consiste en el **estudio de la incertidumbre** dentro de un problema dado.

8.3 Monte Carlo para el cálculo de áreas y volúmenes

Una de las aplicaciones del método es el cálculo de integrales definidas con base en la ley de los grandes números y en la ley del límite central. Vamos a calcular el valor de la integral de una cierta función $f : \mathbb{R}^n \rightarrow \mathbb{R}$, bajo una región $\Omega \subset \mathbb{R}^n$. Para calcular la integral deberemos realizar los pasos siguientes:

1. Generar un punto x de forma uniforme y aleatoria en una región conocida S y un valor aleatorio u uniforme en $I = f(\Omega)$ de forma que $S \times I \subset \Omega \times f(\Omega)$. Usualmente se considera un hiperpoliedro que contenga a Ω . De esta forma, los puntos se generan siguiendo una distribución uniforme en el intervalo $[x_{i,-}, x_{i,+}]$.
2. Calcular el valor de la función $y = f(x)$.
3. Comparar el valor y con el valor generado aleatoriamente u . Si $u < y$ entonces lo contamos como un éxito. Si no, lo contaremos como un fallo.
4. repetimos los pasos 1 a 3 hasta llegar a la precisión deseada. El valor de la integral será la relación:

$$I = V(S) \frac{n_e}{n}.$$

Dónde $V(S)$ es la medida (o volumen) de $S \times I$, n_e es el número de éxitos y n el número de iteraciones que hemos realizado.

Como ya hemos visto, la realización de este experimento se corresponde con una distribución binomial. Con un número suficiente de repeticiones, sabemos que la distribución binomial tiende a una normal. Así pues, podemos utilizar intervalos de confianza para conocer el error que estamos cometiendo. Podemos calcular el tamaño muestral igualando un error ε fijado a priori con la estimación obtenida del intervalo de confianza. Una vez alcancemos una estimación por debajo del rango fijado, podemos concluir que hemos alcanzado el valor deseado para la integración.

Ejercicio 1. Calcula el volumen delimitado por la región del primer cuadrante $(x, y) \in [0, \pi]^2$ y la superficie:

$$f(x, y) = x + y + \sin(xy) + \cos^2(x) + 2.$$

Determinar un intervalo de confianza para el valor de la integral.

Para resolver el problema necesitamos conocer el un entorno 3-dimensional que contenga la región de la que queremos calcular el volumen. Las dos primeras dimensiones se corresponderán con las variables x e y . La tercera variable se corresponde con la altura dentro de la región.

Necesitamos conocer una cota superior de la función. No es necesario que la cota sea estricta. Así pues, vamos a acotar cada uno de los posibles términos:

$$x + y + \sin(xy) + \cos^2(x) + 2 \leq \pi + \pi + 1 + 1 + 2 = 2\pi + 4.$$

Así pues, generaremos tres variables aleatorias uniformes:

- ▶ La primera para la variable x en el intervalo $[0, \pi]$.
- ▶ La segunda para la variable y en el intervalo $[0, \pi]$.
- ▶ La tercera para la variable z en el intervalo $[0, 2\pi + 4]$.

A continuación, evaluamos $f(x, y)$ y comparamos con el valor de z . Si $z < f(x, y)$ añadimos un éxito, si no, no sumamos ningún valor. Podemos emplear el código siguiente:



```
ni=0
n=10000
for i=1:n
    x=rand([1,3]);
    x(1)=x(1)*pi;
    x(2)=x(2)*pi;
    x(3)=x(3)*(2*pi+4);
    fx=x(1)+x(2)+sin(x(1)*x(2))+(cos(x(1)))^2+2;
    if(x(3)<fx)
        ni=ni+1;
    end
end

vol=pi*pi*(2*pi+4)*ni/n
```

La Figura 3 nos muestra una visualización de una ejecución del método de Monte Carlo para calcular dicha integral, debemos tener en cuenta que cada ejecución nos dará un conjunto de puntos distinto. En la gráfica podemos ver la superficie $f(x, y)$ dibujada junto con los 100 primeros puntos generados aleatoriamente. Los puntos rojos son los que quedan por encima de la gráfica y no los consideramos, mientras que los puntos verdes son los éxitos que computamos.

En el gráfico de la izquierda de la Figura 4 podemos ver cómo, a medida que aumentamos el número de puntos que usamos para calcular la integral, nos vamos acercando al resultado final. En esta simulación hemos alcanzado un resultado de 60.24.

Nos queda por comprobar cuál ha sido el error que hemos cometido. Podemos calcular la integral definida de forma numérica. Para ello, Matlab tiene implementados algunos métodos numéricos. Podemos ejecutar la función:

```
q = ...
    integral2(@(X,Y)X+Y+sin(X.*Y).^2+(cos(X)).^2+2,0,pi,0,pi)
```

De esta forma, obtendremos que la integral es 60.23. Un valor muy cercano al que hemos encontrado con el método de Monte Carlo.

Hemos visto que usar el método de Monte Carlo resulta lento si queremos mejorar mucho la precisión. En concreto, nos encontramos en un orden de $1/\sqrt{n}$. Esto es, que el error cometido se reduce en $1/\sqrt{n}$ cuando multiplicamos por n el número de puntos empleado. Podemos visualizar este hecho en el gráfico derecho de la Figura 4. En ella se muestra para cada iterado el error cometido. Tanto el error como el número de iterados están en escala logarítmica. Esto implica que deberíamos ver el error en la gráfica decreciendo como una recta de pendiente $-1/2$. La recta roja que se ve en dicho gráfico tiene dicha pendiente. Podemos ver, como en efecto, el error decrece, con ciertas fluctuaciones, a la velocidad esperada.

Al calcular una integral, queremos saber si nos estamos acercando al valor deseado y cuál es el valor estimado en la aproximación. Como sabemos que los datos se distribuyen de forma normal, podemos repetir el experimento N veces y calcular el intervalo de confianza de los valores I_i . El algoritmo seguirá las líneas siguientes:

- Calculamos N veces el valor de la integral. De forma que obtenemos los valores I_i .
- Como los valores I_i están distribuidos siguiendo una distribución normal, podemos

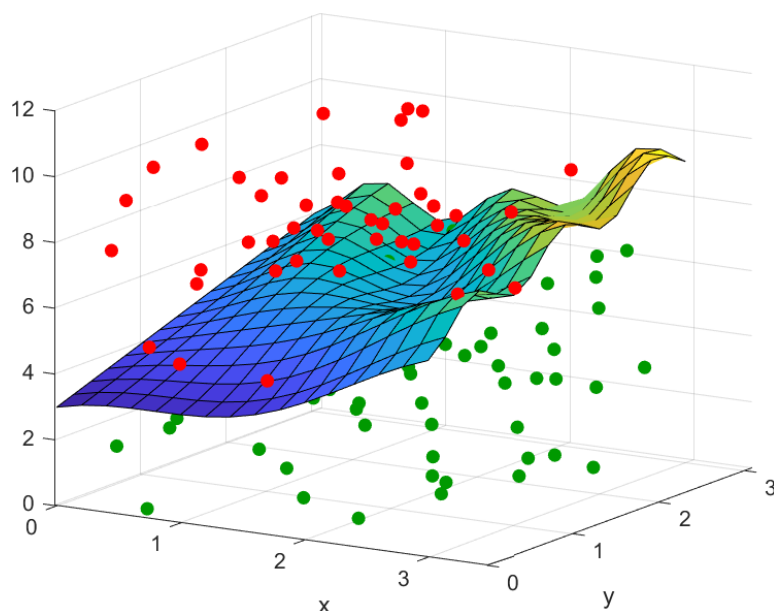


Figura 3: Función $f(x, y)$ junto con los valores generados para obtener la aproximación a la integral mediante el método de Monte Carlo.

calcular el intervalo de confianza para la integral como:

$$I_c = \left[\bar{I} - t_{\alpha/2, N-1} \frac{s(I)}{\sqrt{N}}, \bar{I} + t_{1-\alpha/2, N-1} \frac{s(I)}{\sqrt{N}} \right].$$

Donde $\bar{I}$ es el promedio de los valores I_i , $s(I)$ es el valor de la desviación estándar y $t_{\alpha,k}$ es el valor crítico para una t -Student con una confianza α y k grados de libertad.

Si la variable `integ` contiene N aproximaciones de la integral, podemos calcular el intervalo de confianza en Matlab usando el código:

```
alpha=0.05
SEM = std(integ)/sqrt(length(integ));
ts = tinv([alpha/2 1-alpha/2], length(integ)-1);
CI = mean(integ) + ts*SEM;
```

Como resultado de este proceso, podemos asegurar con un 95 % de confianza que el valor de la integral estará en el intervalo $[60.1706, 60.3702]$, que, en efecto, incluye el valor que conocemos verdadero.

Como hemos podido ver en el ejercicio anterior, los métodos de Monte Carlo son lentos debido al factor $1/\sqrt{n}$. Así pues, para conseguir mejorar el método debemos o bien ampliar el número de iterados, o buscar alguna estrategia para disminuir el orden de convergencia.

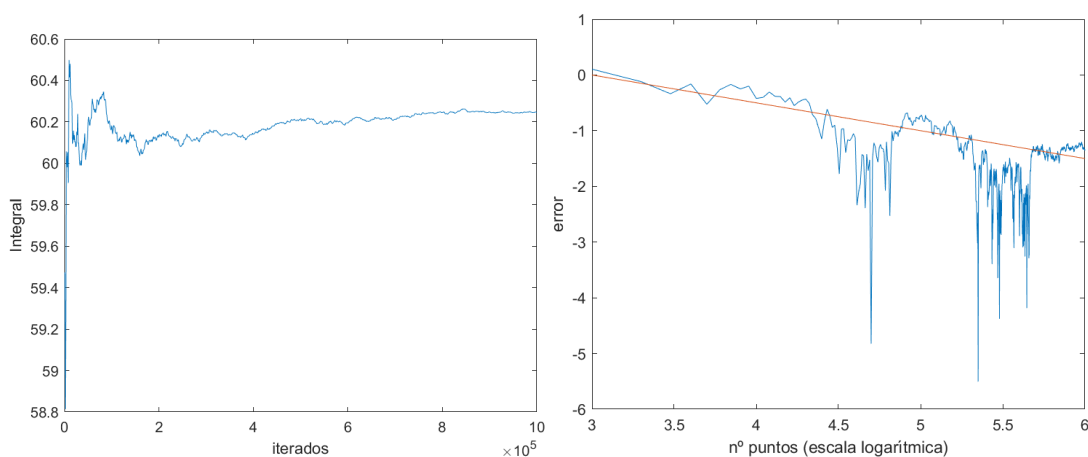


Figura 4: Izquierda: Evolución del valor de la integral calculada mediante el método de Monte Carlo a medida que añadimos más iterados. Derecha: Error cometido en cada iterado (azul) junto con la recta de decrecimiento esperado (rojo).

En el vídeo “Cálculo de integrales mediante Monte Carlo” veremos ejemplos del cálculo de una integral en dimensión alta empleando los métodos de Monte Carlo.



Accede al vídeo: Cálculo de integrales mediante Monte Carlo

Métodos cuasi-Monte Carlo

Una estrategia que se suele emplear para mejorar los métodos de Monte Carlo consiste en usar métodos de generación de números aleatorios que den secuencias que llenen de forma densa el espacio. Con esto conseguimos que el espacio de integración se llene de forma densa y se mejore la precisión del método. Los **métodos de cuasi-Monte Carlo** consisten en cambiar el generador de números aleatorios por otro generador que de valores que, aunque no sean aleatorios, tengan propiedades parecidas.

Para desarrollar estas secuencias emplearemos las **sucesiones de baja discrepancia**. Este tipo de sucesiones se caracterizan por rellenar el espacio de forma que todas las regiones del espacio que estamos cubriendo queden uniformemente rellenas. De forma coloquial podemos pensar en que no dejan agujeros. Existen varios métodos para desarrollar este tipo de sucesiones. Vamos a ver algunos de ellos.

- La sucesión de Halton. El primero de estos métodos consiste en considerar un número primo p_i distinto para cada dimensión de nuestro espacio. A continuación se construye la sucesión de las fracciones generadas con denominador p_i^k para tantos k como sea posible.

Ejemplo 1. Construcción de una sucesión de Halton

Consideramos $p = 2$, entonces empezamos a construir por las fracciones con denominador 2, luego con denominador 4, denominador 8, etc. Debemos tener en cuenta que no repetiremos un mismo valor, de forma que, nos saltaremos el valor $2/4$, el valor $2/8$, etc. La secuencia nos quedará:

$$\frac{1}{2}, \frac{1}{4}, \frac{3}{4}, \frac{1}{8}, \frac{3}{8}, \frac{5}{8}, \frac{7}{8}, \dots$$

Si realizamos la misma acción para cada dimensión con un primo distinto, obtendremos n secuencias de valores que cubren de forma uniforme el espacio. Con esto

conseguiremos recubrir de forma eficiente todo el espacio seleccionado. En la Figura 5 podemos ver la comparación entre emplear valores aleatorios (derecha) o usar una sucesión de Halton (izquierda). Podemos ver como los puntos generados por la sucesión aleatoria se concentran en algunos puntos y dejan zonas en blanco. Sin embargo, la sucesión de Halton cubre el espacio con menos solapamientos y espacios en blanco.

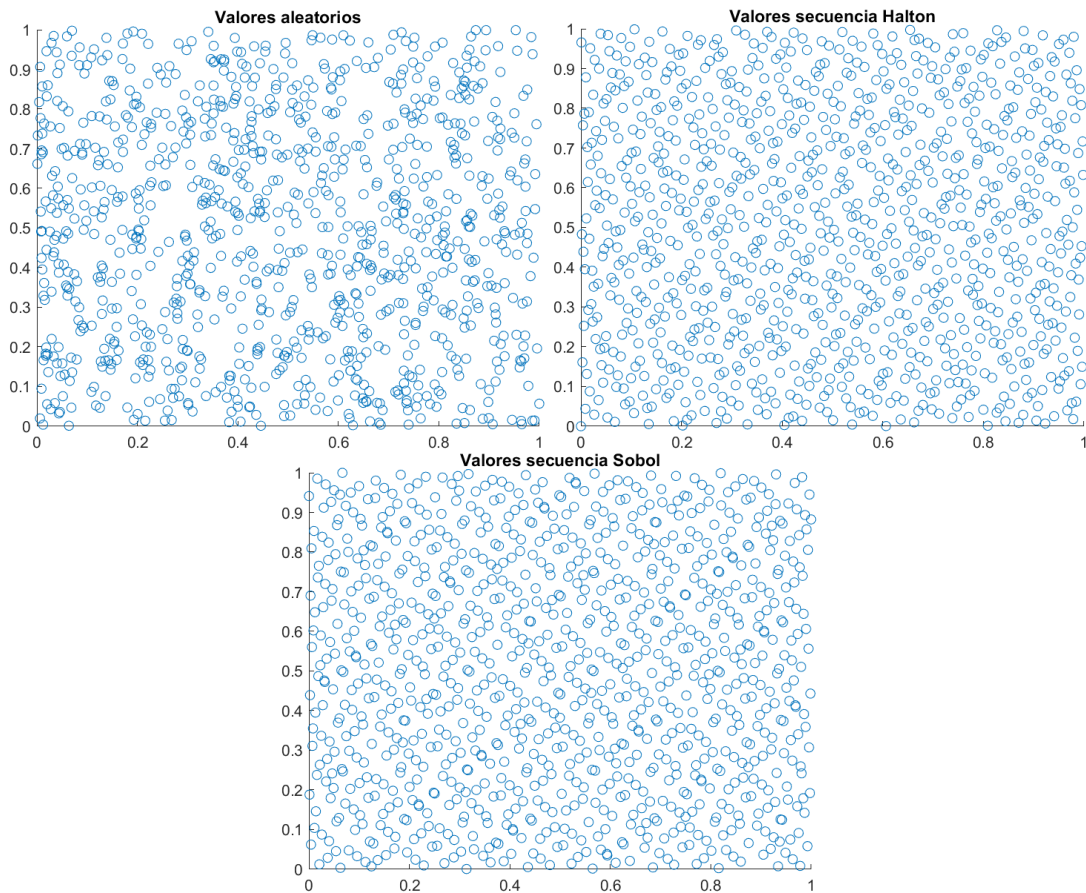


Figura 5: Izquierda: cubrimiento del cuadrado $[0, 1]^2$ por valores aleatorios. Derecha: cubrimiento del mismo cuadrado siguiendo una secuencia de Halton. Inferior: Recubrimiento del mismo cuadrado usando sucesiones de Sobol saltando los primeros 1000 valores.

Podemos generar valores siguiendo una secuencia de Halton en matlab empleando el código siguiente:

```
net(haltonset(2),1000);
```

Con esto, generaremos 1000 puntos en un espacio de dimensión 2 usando la sucesión de Halton.

- Sucesión de Sobol. El segundo de los métodos consiste en generar valores siguien-

do una sucesión de Sobol. Este método consiste en generar una ristra de valores de la forma $\frac{k}{2^n}$, para $k \in \{0, \dots, 2^{n-1}\}$. Tomaremos los valores cuasi-aleatorios a partir de una suma de una combinación de estos. A nivel práctico, lo que hacemos es calcular la representación binaria de los diferentes valores posibles.

Igual que ocurre con la sucesión de Halton, podemos calcular los valores de una sucesión de Sobol en Matlab. Para ello, emplearemos el código:

```
net(sobolset(2),1000);
```

Con esto, generaremos 1000 puntos en un espacio de dimensión 2 usando la sucesión de Sobol. La gráfica inferior de la figura 5 nos muestra los puntos generados de forma cuasi-aleatoria empleando una sucesión de Sobol.

Cuando se emplean este tipo de sucesiones, suele ser recomendable saltar los primeros valores de la sucesión. Estos valores tienden a agruparse con valores calculados posteriormente. Este hecho se puede incorporar a Matlab usando la opción:

```
net(sobolset(2,'Skip',1e3),1000);
```

Si repetimos el ejercicio del cálculo de la integral usando una sucesión de Halton con valores primos 2, 3 y 5 obtenemos la gráfica de la figura 6 En ella podemos observar como el error decrece más rápidamente.

8.4 Monte Carlo para el estudio de incertidumbre

La segunda aplicación que veremos del método de Monte Carlo consiste en realizar un mismo experimento multitud de veces para conocer el efecto de la incertidumbre en el modelo.

Bajo esta aproximación, se realiza el un determinado experimento que dependa del azar n veces. En cada nueva repetición debemos anotar el resultado final del experimento. Al finalizar el estudio dispondremos de una colección de datos amplia que nos permitirá obtener resultados estadísticos.

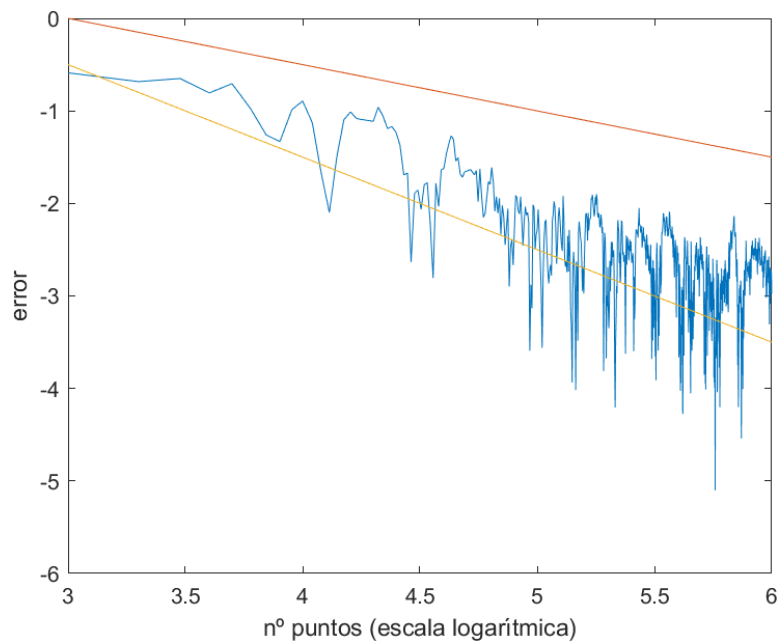


Figura 6: Evolución del error usando una sucesión de Halton en el cálculo de la integral del ejercicio 1.

Este tipo de simulaciones se emplean en simulaciones en las que interviene el azar o en las que se desconocen las condiciones de partida. Entre otros, algunos ejemplos son:

- **Optimización.** En este tipo de problemas buscamos encontrar mínimo o máximo valor a una cierta función $f(\vec{x})$. Esta función se conoce como función objetivo. Podemos realizar exploraciones aleatorias del experimento para obtener candidatos al óptimo. Usualmente, suelen combinarse con otros métodos para afinar la búsqueda o usarse como paso inicial para utilizar el resultado de la optimización por Monte Carlo como semilla inicial.
- **Resolución del problema inverso.** A menudo, nos encontramos ante un sistema que del que no conocemos la distribución inicial pero hemos recolectado datos. Mediante la simulación de Monte Carlo, se puede realizar una exploración del sistema para encontrar la distribución de probabilidad que nos permitirá obtener los resultados más parecidos al sistema real. También entrarán dentro de este tipo de experimentos, aquellos en los que deseamos encontrar los parámetros necesarios dentro del sistema. Mediante la distribución aleatoria de los parámetros, podemos encontrar cuáles son los más interesantes o cercanos a la realidad.

Ejemplo 2. Estudio de retenciones por un semáforo

Consideramos una avenida de una gran ciudad. Dicha avenida está regulada mediante un semáforo al que llegan vehículos cada cierto tiempo siguiendo una cierta ley de probabilidad. Se tiene el control de la proporción de tiempo que el semáforo está activo (en verde) e inactivo (en rojo). Podemos emplear una simulación de Monte Carlo para responder a algunas de estas preguntas:

- ▶ ¿Cuál será la variación del tiempo de espera si la afluencia de tráfico al cruce se incrementa o reduce en un porcentaje dado?
- ▶ ¿Qué sucede si añadimos uno o más semáforos al sistema?
- ▶ ¿Cuáles son los elementos que provocan una mayor congestión del sistema? ¿Cómo se puede evitar dicha congestión?
- ▶ ¿Cómo se puede minimizar el tiempo de espera de los vehículos?

Para responder a estas preguntas deberemos preparar un modelo que regule el tráfico. A continuación, deberemos tomar los valores que queremos estudiar, por ejemplo, el tiempo de afluencia entre vehículos y realizar variaciones de dicho tiempo. Así, conseguiremos determinar la afectación para dichas entradas y obtener resultados sobre los tiempos óptimos para la relación de tiempos de activación y desactivación del semáforo.

Ejercicio 2. Considera el modelo desarrollado en el tema anterior del movimiento de una bala de cañón con la influencia del viento. Determina cuál es la posición esperada de colisión con el suelo.

En el vídeo “Estudio de la incertidumbre mediante Monte Carlo” veremos cómo resolver este ejercicio en todo detalle.



Accede al vídeo: Estudio de la incertidumbre mediante Monte Carlo

8.5 Referencias bibliográficas

Metropolis, N. and Ulam, S. (1949). The monte carlo method. *Journal of the American Statistical Association*, 44(247):335–341. PMID: 18139350.